

Real-Time Network Intrusion Detection and Monitoring System

Snort 3 + Scapy + SQLite + Streamlit

Updated Project Documentation - Dashboard v4.0

Field	Value
Project	Real-Time Network Intrusion Detection System Dashboard
Latest dashboard version	dashboard-v4.0
Author	Maheen Nadeem
Main technologies	Snort 3, Scapy, SQLite, Streamlit, Python, Plotly, tcpreplay
Latest update focus	Enriched alert JSON, PCAP replay statistics, 10,000-alert retention, selected enriched JSON display, and clearer replay-to-alert workflow

1. Project Description

The Real-Time Network Intrusion Detection and Monitoring System monitors live traffic in a VirtualBox lab, detects suspicious activity using Snort 3 Community Rules, and visualizes packet and alert data through an interactive Streamlit dashboard. The system combines live packet capture, signature-based detection, SQLite storage, automatic refresh, rule-based alert explanation, and custom PCAP replay testing.

Dashboard v4.0 extends the previous dashboard by adding a Total Raw Alerts metric, increasing alert retention to the latest 10,000 alerts, displaying a selected enriched alert JSON record, and adding a PCAP replay statistics section. It also introduces an enriched JSON export that combines recent database alerts with rule documentation for dashboard display and future LLM/RAG-based explanation work.

2. Objectives

- Capture live traffic from the VirtualBox host-only lab network.
- Detect suspicious activity using Snort 3 Community Rules.
- Store packet and alert metadata in SQLite using rolling-window cleanup.
- Separate normal traffic visibility from alert investigation in the dashboard interface.
- Display packet trends, services, alert signatures, priorities, source/destination IPs, and detailed alert fields.
- Preprocess and normalize Snort rule documentation, CVEs, MITRE mappings, references, and content patterns.
- Export rule documentation as a SID-keyed JSON lookup dataset.
- Generate an enriched alert JSON snapshot by joining recent alert records with matching rule documentation.
- Create and replay a compact multi-attack PCAP for repeatable testing and dashboard demonstration.
- Maintain GitHub version history using dashboard version tags.

3. System Components

Component	Role	Purpose
Kali Linux VM - 192.168.56.10	Traffic generator / attacker	Generates normal and suspicious traffic, including scans, web requests, DVWA attacks, sqlmap traffic, and PCAP replay

		using tcpreplay.
Ubuntu Victim VM - 192.168.56.20	Target machine	Receives traffic and hosts test services such as Apache/HTTP, SSH, and DVWA.
Ubuntu Snort/Dashboard VM - 192.168.56.30	Monitoring machine	Runs Snort 3, Scapy packet collector, alert collector, SQLite database, rule documentation dataset, enriched JSON export, and Streamlit dashboard.
Windows Host	Access gateway and repository host	Uses VirtualBox NAT forwarding and Windows portproxy for dashboard access and stores the GitHub working repository.

4. Snort Overview

Snort is a network intrusion detection and prevention system. In this project, Snort 3 works as the signature-based detection engine. It observes network traffic, compares packets and flows against the loaded Snort Community Rules, and generates alerts when traffic matches an enabled rule.

Snort does not generate one alert per packet. It only generates alerts for traffic that matches a rule or enabled inspector event. This is why a PCAP with many packets may produce a much smaller number of alerts.

5. Current Architecture and Workflow

- Kali sends normal or attack traffic to the Ubuntu Victim VM over the host-only lab network.
- The Ubuntu Snort/Dashboard VM observes traffic on interface enp0s8.
- The Scapy packet collector captures general packet metadata and stores it in the packets table.
- Snort analyzes traffic against the local Community Rules file and writes JSON alerts when rules match.
- Snort alert output is appended to /home/maheen/snort_logs/alert_json.txt.
- The alert collector reads alert_json.txt, parses JSON alert fields, and inserts them into the alerts table.
- The Streamlit dashboard reads SQLite and refreshes automatically to display packet visibility and alert analytics.
- The export script joins database alerts with rule documentation to create enriched_snort_alerts_with_rule_docs.json.
- The dashboard displays selected enriched alert JSON for analyst-friendly alert context.

Kali VM -> Ubuntu Victim VM

\ /
\ /

Ubuntu Snort/Dashboard VM

Snort 3 + Scapy + SQLite + Streamlit

Rule documentation JSON + enriched alert JSON

6. PCAP Replay and Live Detection Flow

Dashboard v4.0 documents the complete replay-to-alert path used for repeatable testing:

Step	Action	Output / Storage
1	Kali replays multi_attack_demo_mtu1500.pcap using tcpreplay on eth1.	Replayed traffic enters the host-only lab network.
2	Ubuntu Victim receives the replayed traffic.	Traffic reaches the target services and DVWA paths originally captured in the PCAP.
3	Snort/Dashboard VM monitors the network on enp0s8.	Traffic is visible to Snort and the Scapy packet collector.

4	Snort compares traffic with Snort Community Rules.	Matching traffic produces JSON alerts.
5	Snort appends JSON alerts to alert_json.txt.	/home/maheen/snort_logs/alert_json.txt stores raw alert history unless cleared.
6	live_alert_collector.py reads alert_json.txt.	Parsed alert fields are inserted into ids_live.db -> alerts table.
7	live_packet_collector.py captures packet metadata.	Packet metadata is inserted into ids_live.db -> packets table.
8	export_enriched_alerts_json.py joins alerts and rule docs.	data/json/enriched_snort_alerts_with_rule_docs.json is created or overwritten as the latest enriched snapshot.
9	Streamlit dashboard reads SQLite and enriched JSON.	Traffic metrics, alert metrics, selected enriched alert JSON, and PCAP statistics are displayed.

PCAP replay -> Snort rule matching -> alert_json.txt -> alerts table -> enriched JSON -> dashboard

7. Remote Dashboard Access

The dashboard is hosted inside the Ubuntu Snort/Dashboard VM. Windows does not host the Streamlit app directly. Windows acts as a forwarding gateway so other reachable systems can open the dashboard using the Windows host IP address.

Client/Mentor System -> Windows Host IP:8501 -> Windows portproxy -> 127.0.0.1:8501 -> VirtualBox NAT Port Forwarding -> Ubuntu VM Streamlit Dashboard

- Streamlit must be running inside the Ubuntu VM.
- VirtualBox NAT forwarding must forward Windows localhost port 8501 to the Ubuntu VM port 8501.
- Windows portproxy must expose the current Windows network IP on port 8501.
- Previously captured SQLite data can still be displayed when live attacks are stopped, but the VM and dashboard must remain online.

8. Database Design

8.1 Live Dashboard Tables

Table	Main Fields	Retention
packets	timestamp, src, dst, protocol, src_port, dst_port, size, info, tcp_flags	Latest 5,000 rows
alerts	timestamp, rule/msg, classification, priority, protocol, src/dst, snort_rule, iface, service, src_addr, src_port, dst_addr, dst_port, direction, pkt_len, action, gid, sid, rev, ttl, tos, tcp_flags, icmp_type, icmp_code, client/server packet and byte counters, raw_json	Latest 10,000 rows

The packets and alerts tables are stored inside the same SQLite database, ids_live.db, but they serve different purposes. The packet collector writes general traffic metadata to the packets table. The alert collector writes Snort detections to the alerts table. Separate rolling limits are used because packet visibility and alert investigation have different storage needs.

8.2 Raw and JSON Storage Files

File	Purpose	Behavior
------	---------	----------

alert_json.txt	Raw Snort JSON alert log.	Append-only during live Snort execution unless manually cleared for a clean test.
rule_docs_preprocessed_by_sid.json	Static Snort rule documentation lookup dataset.	Contains processed rule documentation keyed by SID or GID/SID.
enriched_snort_alerts_with_rule_docs.json	Latest enriched alert snapshot.	Generated by joining current database alerts with rule documentation. It is overwritten on export rather than appended.

9. Collector Architecture

Collector	Input	Output	Purpose
live_packet_collector.py	Network interface enp0s8	ids_live.db -> packets table	Provides packet-level traffic visibility for the Network Traffic Dashboard.
live_alert_collector.py	alert_json.txt generated by Snort	ids_live.db -> alerts table	Stores Snort alert events and rich alert fields for the Alert Dashboard.

The collectors run in parallel. Packet collection and alert collection are related but not identical: many packets may be captured, but only some packets or flows produce Snort alerts. A packet count should therefore not be interpreted as an alert count.

10. Rule Documentation Knowledge Database

The project includes a cleaned and normalized Snort rule documentation knowledge database. It is separate from the rolling-window packets and alerts tables and is used for rule lookup, research, dashboard explanation, and future LLM/RAG work.

Table / Dataset	Purpose	Rows / Relationship
rules	Main parent table with one row per Snort rule, including rule identity, header fields, flow, service, metadata, and rule_text.	4,017 rows; primary key is gid + sid.
rule_documentation	Snort.org documentation fields such as category, explanation, what to look for, known usage, false positives, rule groups, and vulnerability text.	4,017 rows; connected to rules by gid + sid.
rule_cves	Stores one CVE per rule instead of comma-separated CVEs.	2,278 rows; many CVEs can belong to one rule.
rule_mitre	Stores MITRE ATT&CK mappings with technique ID, tactic, and technique name.	207 rows; a rule can map to multiple techniques.
rule_references	Stores external references such as CVE, URL, Bugtraq, Nessus, and other links.	6,735 rows; a rule can have many references.
rule_content_matches	Stores Snort content options separately, including order and negation.	7,413 rows; a rule can contain many content patterns.
rule_docs_preprocessed_by_sid.json	Fast JSON lookup used by the dashboard and export script.	Keyed by SID and GID/SID variants.

11. Enriched Alert JSON Export

Dashboard v4.0 adds an enriched alert JSON export. This file combines recent alerts from the SQLite alerts table with matching rule documentation from rule_docs_preprocessed_by_sid.json. The join is performed using GID and SID.

Input	Role
-------	------

ids_live.db -> alerts table	Provides recent parsed Snort alert records and dashboard alert fields.
data/json/rule_docs_preprocessed_by_sid.json	Provides rule explanation, category, CVEs, MITRE mapping, references, and original rule text.
scripts/export_enriched_alerts_json.py	Exports the latest enriched snapshot to JSON.
data/json/enriched_snort_alerts_with_rule_docs.json	Combined alert + rule documentation artifact used by the dashboard and future LLM/RAG work.

The enriched JSON is intentionally treated as a snapshot. It is overwritten each time the export script runs. This avoids duplicate records and prevents the file from becoming too large for dashboard or LLM use. Long-term raw alert history remains in alert_json.txt if it is not cleared.

12. Dashboard Components Implemented in v4.0

Dashboard Panel	Status	Purpose
Version and page sidebar	Implemented	Shows dashboard version and page navigation.
Network Traffic Dashboard	Implemented	Separates normal packet and host visibility from alert analysis.
Alert Dashboard	Implemented	Focuses on Snort detections, alert details, and rule-based investigation.
Total Raw Alerts metric	Added in v4.0	Counts raw JSON alert lines from alert_json.txt, so the total can grow beyond the database retention window.
Alert retention increase	Added in v4.0	alerts table retention increased from latest 1,000 rows to latest 10,000 rows.
Detailed Recent Snort Alerts	Implemented	Displays alert fields such as rule ID, classification, priority, protocol, ports, direction, TTL, TCP flags, ICMP type/code, and flow counters.
Selected Enriched Alert JSON	Added in v4.0	Displays the combined alert and rule documentation JSON for the selected alert.
PCAP replay statistics table	Added in v4.0	Summarizes replayed packets, replay speed, duration, successful packets, raw alerts, and alert rate.
Ruleset Status	Implemented	Shows Snort version, rules file path, enabled rule count, file timestamp, size, and SHA-256 hash.

13. Custom Multi-Attack PCAP Generation

A custom PCAP was created because many public IDS datasets are very large and impractical for the available lab storage. The custom PCAP provides compact and repeatable traffic for demonstrations and dashboard testing.

- Gobuster directory enumeration.
- Nikto web vulnerability scanning.
- DVWA attacks including SQL injection, XSS, and file inclusion.
- sqlmap SQL injection testing.
- Path traversal and file inclusion attempts.
- Command injection payloads.
- Brute-force login attempts.
- curl and manual HTTP traffic.

PCAP Metric	Value
-------------	-------

PCAP file	multi_attack_demo_mtu1500.pcap
Packets replayed	135,292
Data size	37.46 MB
Replay speed	800 packets/second
Replay duration	169.11 seconds
Successful packets	135,292
Failed packets	0
Raw Snort alerts generated	786
Alert rate	0.58%
Approximate alert ratio	1 alert per 172 packets

Packets are not equal to alerts. Most replayed packets are normal handshakes, ACKs, HTTP responses, or ordinary traffic. Snort only generates alerts when traffic matches enabled rules.

14. Top Alerts Observed During PCAP Replay

Alert Signature	Count	Interpretation
SERVER-WEBAPP Compaq Insight directory traversal	320	Directory traversal / web reconnaissance pattern. Likely caused by web scanner probes.
SERVER-WEBAPP iChat directory traversal attempt	215	Directory traversal / legacy web exploit probe. Likely caused by Nikto or similar web vulnerability scanning.
OS-MOBILE Apple iPhone User-Agent detected	50	Mobile-like User-Agent fingerprinting or scanner header variation, not necessarily a direct exploit.
OS-MOBILE Android User-Agent detected	33	Mobile-like User-Agent fingerprinting or scanner header variation.
SERVER-WEBAPP remote include path attempt	15	Remote file/path inclusion attempt associated with web application attack testing.

15. Running the Project

15.1 Start Snort

```
mkdir -p /home/maheen/snort_logs
sudo snort -q \
-c /usr/local/etc/snort/snort.lua \
-R /usr/local/etc/snort/rules/snort3-community.rules \
-i enp0s8 \
-A alert_json \
--lua "HOME_NET='192.168.56.20/32'; EXTERNAL_NET='192.168.56.10/32'; alert_json={fields='timestamp iface proto service src_addr src_port dst_addr dst_port dir pkt_len action msg class priority rule gid sid rev'}" \
2>/tmp/snort_json_errors.txt | tee -a /home/maheen/snort_logs/alert_json.txt
```

15.2 Start Alert Collector

```
cd /home/maheen/ids_streamlit_dashboard
python3 live_alert_collector.py
```

15.3 Start Packet Collector

```
cd /home/maheen/ids_streamlit_dashboard
sudo python3 live_packet_collector.py
```

15.4 Start Dashboard

```
cd /home/maheen/ids_streamlit_dashboard
streamlit run dashboard.py --server.address 0.0.0.0 --server.port 8501 --server.headless true
```

15.5 Replay Custom PCAP from Kali

```
sudo tcpreplay -i eth1 --loop=1 --pps=800 ~/nids-pcaps/scanner-test/clean/multi_attack_demo_mtu1500.pcap
```

15.6 Generate Enriched Alert JSON

```
cd /home/maheen/ids_streamlit_dashboard
python3 scripts/export_enriched_alerts_json.py
```

16. GitHub Repository and Versioning

Version	Description
dashboard-v1.0	First complete Streamlit IDS dashboard release.
dashboard-v2.0	Updated dashboard with Snort rule datasets, helper scripts, and documentation.
dashboard-v3.0	Page navigation, clickable alert-row JSON rule lookup, metric cards, compact charts, and cleaner rule documentation display.
dashboard-v4.0	Total Raw Alerts metric, 10,000 alert retention, enriched alert JSON export/display, custom PCAP replay statistics, and documented PCAP replay-to-alert workflow.

```
git tag -a dashboard-v4.0 -m "Dashboard version 4.0 - enriched alert JSON, PCAP replay statistics, and 10000 alert retention"
```

17. Limitations

Limitation	Impact	Mitigation / Future Work
Rule/signature dependency	Snort alerts only when traffic matches loaded rules or enabled inspector events.	Keep rules updated and validate detections using known lab traffic.
Ruleset version dependency	Different rule revisions may detect or classify traffic differently.	Record Snort version, rules file hash, enabled rule count, and file timestamp.
SQLite backend	Good for lab scale but not ideal for enterprise-scale logging.	Use PostgreSQL, Elasticsearch, or a SIEM for larger deployments.
Collectors must run	Alerts do not appear in the dashboard until live_alert_collector.py inserts them into SQLite.	Add systemd services or a collector health panel.
Limited alert-packet correlation	Alerts and packets are stored separately.	Correlate by timestamp, protocol, IPs, ports, and direction.
Encrypted traffic	HTTPS/TLS payloads cannot be deeply inspected without decryption.	Use metadata, endpoint logs, proxy logs, or approved TLS inspection.
Large alert volumes	Dashboard performance may degrade with very large datasets.	Use rolling windows, pagination, and larger backends.

18. Recent Work Log Updates

Date	Description
------	-------------

17-07-2026	Improved attack traffic generation for IDS testing. Explored tools such as Gobuster, Nikto, sqlmap, Packet Sender, tcpreplay, and iperf3. Reviewed public IDS PCAP datasets and determined that many were too large for the current lab setup.
20-07-2026	Created a custom multi-attack PCAP using Gobuster, Nikto, DVWA, sqlmap, XSS payloads, file inclusion, command injection, brute-force attempts, and manual HTTP traffic. Cleaned the PCAP for MTU-safe replay and recorded replay statistics: 135,292 packets, 37.46 MB, 800 pps, 169.11 seconds, and 0 failed packets. Snort generated about 786 raw alerts.
21-07-2026	Updated dashboard and alert storage design. Increased alerts table retention to 10,000, added Total Raw Alerts from alert_json.txt, created enriched JSON export from ids_live.db alerts plus rule_docs_preprocessed_by_sid.json, updated selected alert display to show combined enriched JSON, added PCAP statistics, and updated documentation/presentation material.

19. Future Enhancements

- Add a collector/database health panel showing collector status, alert_json.txt status, and database counts.
- Add alert-to-packet correlation using timestamp, protocol, IPs, ports, and direction.
- Add exportable reports for alerts and packet summaries.
- Add email, Slack, or dashboard notifications for priority 1 and priority 2 alerts.
- Add optional anomaly detection to complement signature-based Snort alerts.
- Add authentication for shared dashboard access.
- Support multiple sensors or larger databases such as PostgreSQL or Elastic.
- Integrate the enriched JSON and normalized rule dataset with an LLM/RAG explanation module.

20. Conclusion

Dashboard v4.0 improves the project by adding raw alert counting, increased alert retention, enriched alert JSON display, and a documented custom PCAP replay workflow. Scapy provides general traffic visibility, while Snort provides rule-based detection. SQLite supports a fast lab-scale dashboard, and the preprocessed rule documentation dataset adds explanation context to alerts.

The project now demonstrates the complete pipeline: attack traffic generation, PCAP replay, Snort rule matching, raw alert logging, SQLite storage, enriched JSON export, and Streamlit dashboard visualization. It is organized for GitHub versioning and future LLM/RAG integration.